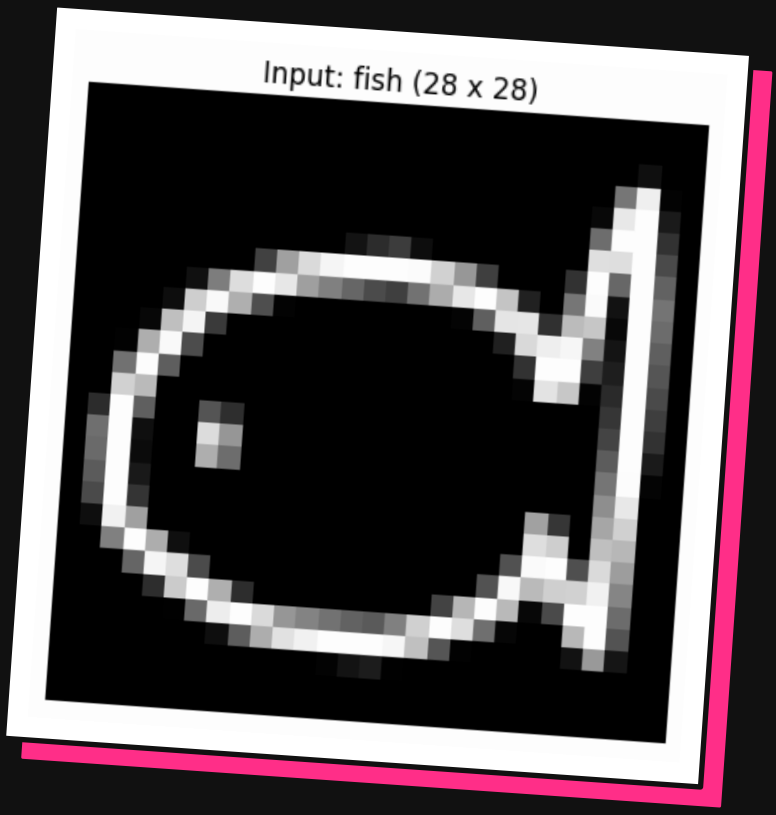
DEEP LEARNING PRESENTATION

RESNET-50

× QUICK, DRAW!

How a 50-layer CNN learns to read doodles: the architecture explained, implemented in PyTorch, and compared with an IEEE 2025 paper.



INPUT: A REAL QUICK, DRAW! FISH

Hrishikesh Gavai
BE-COMP-A · Roll No. 17

WHAT IS RESNET-50?

A **50-layer convolutional neural network (CNN)** from Microsoft Research (He et al., 2015). Its big idea, the **skip connection**, let networks go far deeper than before without getting worse.

50
layers deep

25.6M
parameters

#1
ImageNet ILSVRC 2015

It won ImageNet 2015 with a 3.57% top-5 error, while using about 5x fewer parameters than VGG-16.

HOW IT COMPARES

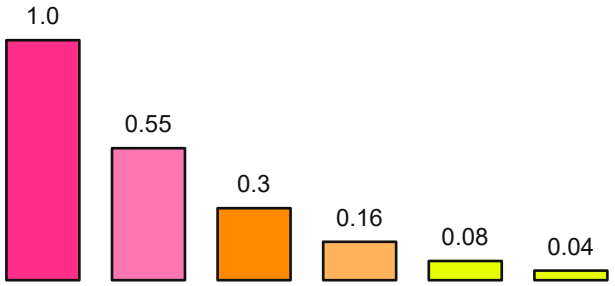
Model	Year	Layers	Params	Limitation / strength
LeNet-5	1998	5	60K	Too shallow for real-world images
AlexNet	2012	8	60M	Shallow; big 11x11 filters
VGG-16	2014	16	138M	Very heavy; going deeper made it worse
ResNet-50	2015	50	25.6M	Deep AND trainable, thanks to skip connections

Deeper than VGG-16, yet lighter: ResNet-50 gets more accuracy per parameter.

THE PROBLEM, AND THE FIX

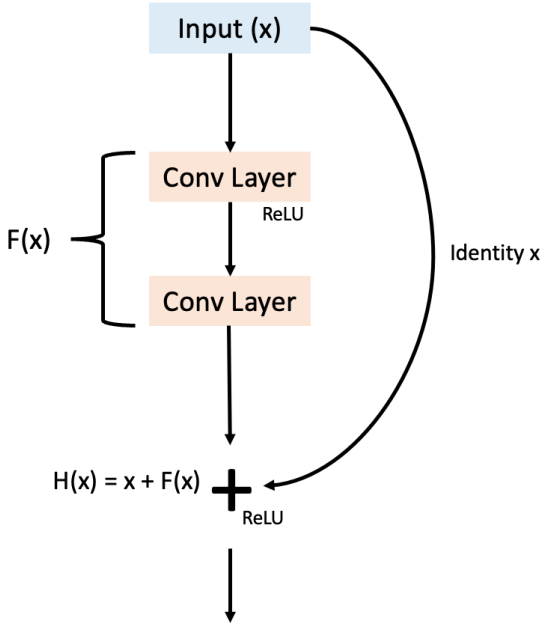
THE PROBLEM: VANISHING GRADIENT

- Training sends an error signal (the gradient) backwards through every layer.
- At each layer it is multiplied by small numbers, so it keeps shrinking.
- After 20+ layers it is almost 0: the early layers stop learning.



gradient size: layer 50 → layer 1

Deeper “plain” CNNs did WORSE.



Residual block: the input x jumps over the conv layers and is added back.

THE FIX: SKIP CONNECTION

$$H(x) = F(x) + x$$

x	the input, copied straight through
F(x)	what the layers learn: only the change
H(x)	the output: input + change

WHY IT WORKS

The problem: the learning signal fades as it travels back through many layers.

The shortcut: skip connections give it a direct path, so it never fades.

Small fixes, not rebuilds: each block only improves what it's given.

Can't get worse: if a block isn't helping, it just passes the input through unchanged.

KEY TERMS: WHAT EACH ONE DOES

CONVOLUTION

Slides a small 3x3 window over the image, multiplying and adding pixels, to light up wherever a pattern (like an edge) appears.

FEATURE MAP

The “heat map” one filter makes: bright where its pattern was found. 64 filters give 64 feature maps (channels).

LAYER

One processing step. Each layer builds on the last: edges → strokes → parts → whole objects.

POOLING

Shrinks each map by keeping only the strongest value in every small patch: half the size, same key info.

BATCH NORMALIZATION

Re-centres and re-scales each layer's outputs so the numbers stay stable; training gets faster and steadier.

RELU

max(0, x): keeps positives, turns negatives into 0. This lets the network learn curved, complex shapes.

BOTTLENECK BLOCK

A 1x1 conv squeezes 256 channels to 64, the costly 3x3 runs on only those 64, then a 1x1 expands back to 256: ~8x less work.

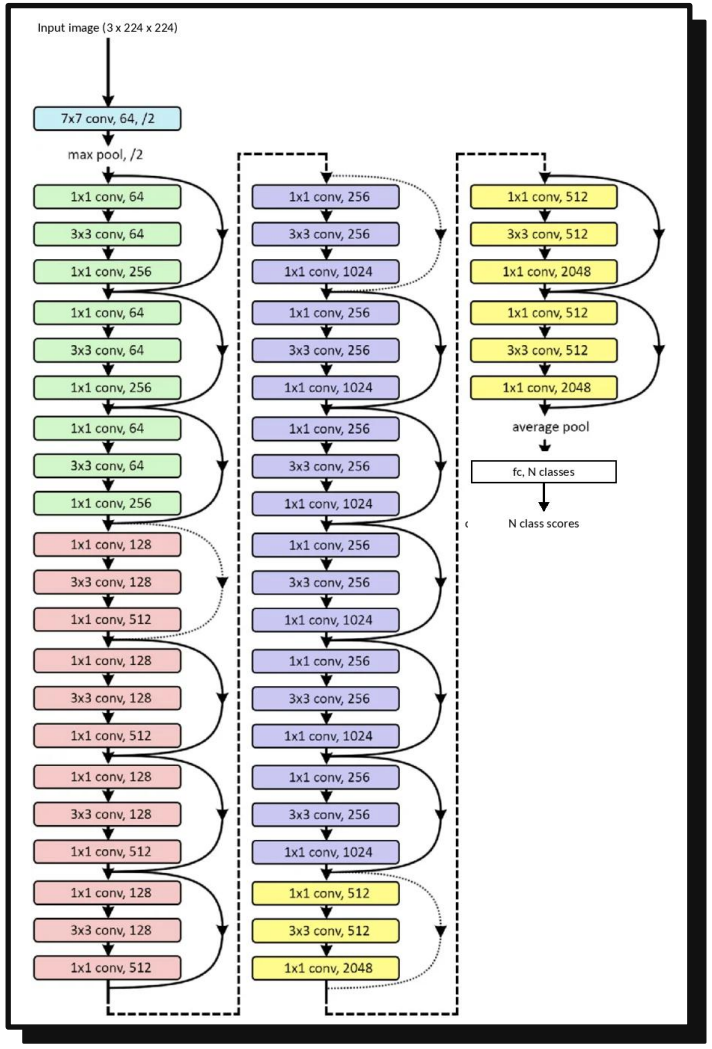
SKIP CONNECTION

Carries a block's input around its layers and adds it to the output, H(x) = F(x) + x: a shortcut for data and gradients.

FULLY CONNECTED (FC)

Links all 2048 features to every category, weighs them up and gives one score per class. Highest score wins.

RESNET-50 ARCHITECTURE



Step	Output
Input	3 × 224 × 224 (gray copied into 3 channels)
conv1 (7×7, stride 2)	64 × 112 × 112
max pool	64 × 56 × 56
Layer 1: 3 bottlenecks	256 × 56 × 56
Layer 2: 4 bottlenecks	512 × 28 × 28
Layer 3: 6 bottlenecks	1024 × 14 × 14
Layer 4: 3 bottlenecks	2048 × 7 × 7
avg pool	2048 values
fc	10 / 20 / 30 / 50 scores (one per category)
argmax	**highest score = predicted category**

ONE BOTTLENECK BLOCK

1x1 conv
SHRINK 256 → 64

→

3x3 conv
LEARN on 64

→

1x1 conv
EXPAND 64 → 256

→

+ X

Why 50? 1 (conv1) + 16 blocks × 3 conv layers + 1 (fc) = 50. Each stage halves the size and doubles the channels: it trades “where” for “what”.

THE DATA: QUICK, DRAW!

Google's online game (2016): you get 20 seconds to draw an object while a neural network guesses it. Every drawing is saved as a 28 × 28 grayscale bitmap (.npy file).

50M drawings

345 categories

15M players

OUR PIPELINE

#	Step	Why
1	**Load 28×28 drawing**	from Google's .npy bitmap files
2	**Resize to 224×224**	the input size ResNet-50 expects
3	**Copy 1 → 3 channels**	pretrained weights expect RGB
4	**Normalize**	ImageNet mean and std
5	**ResNet-50**	outputs one score per category

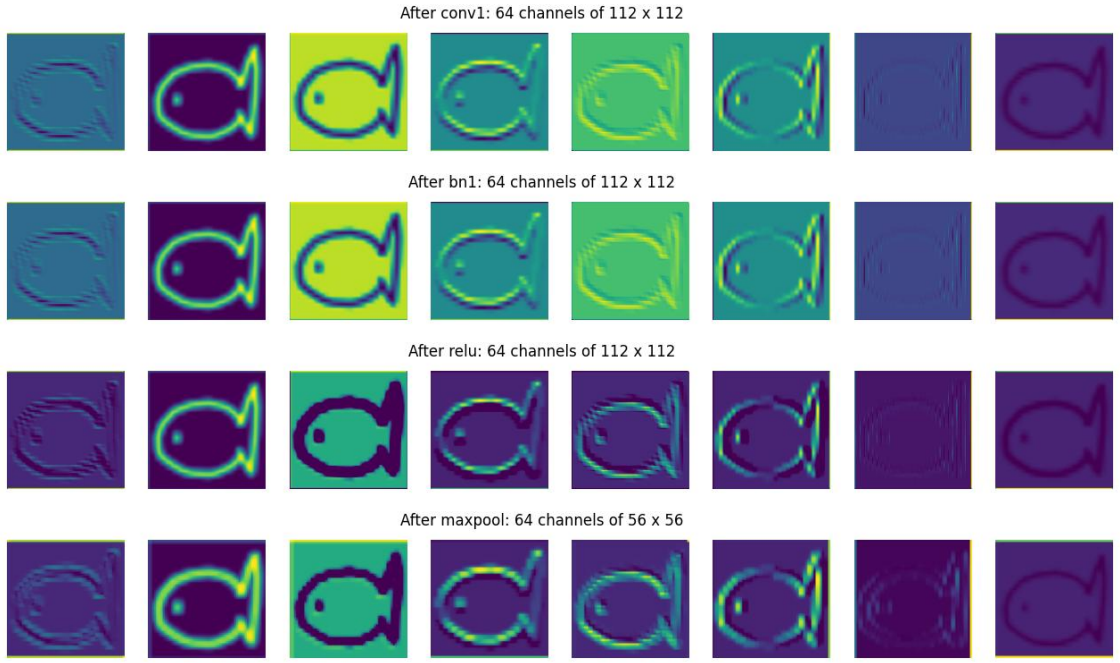
SAME SETUP AS THE PAPER

Setting	Value
Categories	10 · 20 · 30 · 50 (from our list of 100)
Drawings per category	1,000 (random sample)
Split	80% train / 10% val / 10% test
Input	224×224, grayscale, normalized
Model	ResNet-50, ImageNet weights*
Optimizer · LR · batch	AdamW · 1×10^{-5} · 64
Loss	Cross-entropy
Epochs	15 (10/20/30) · 20 (50)
Augmentation	Horizontal flip + rotate ±10°
Hardware	Google Colab T4 GPU, PyTorch

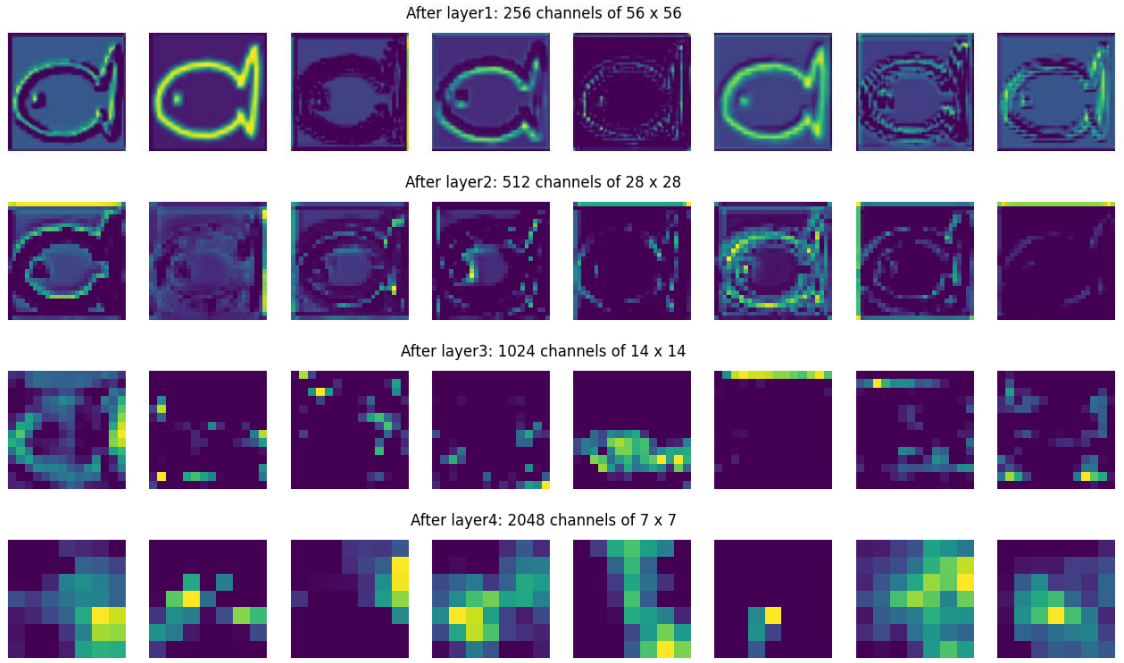
**The paper states ImageNet pretraining for ViT; we assume the same for ResNet-50.*

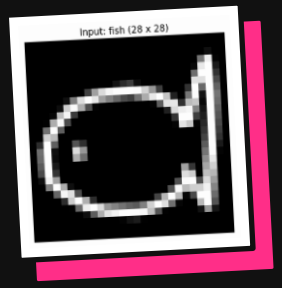
ONE FISH, LAYER BY LAYER

STEM



BOTTLENECK STAGES





PREDICTION: FISH ✓

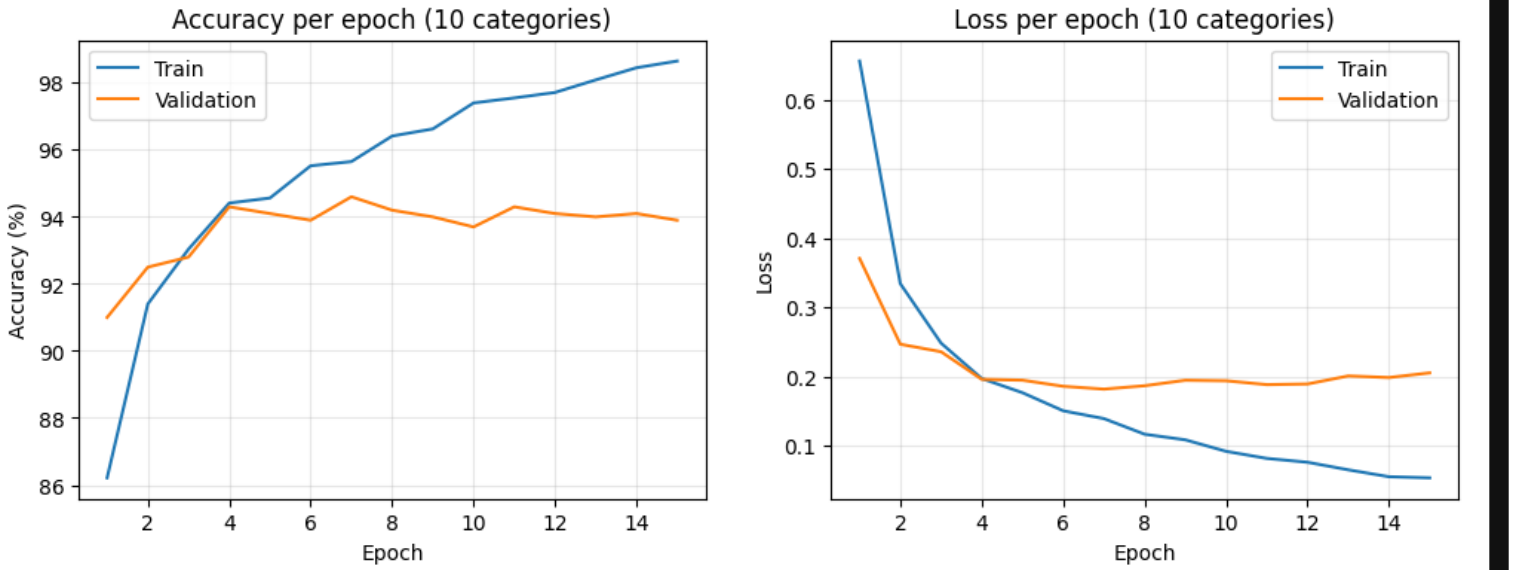
input 28×28 → 224×224

Early layers see edges
conv1 → layer1: the outline, eye and tail strokes.

Deep layers see shapes
layer3 → layer4: abstract "fish-ness" blobs.

Smaller, but deeper
112×112 → 7×7 in size; 64 → 2048 channels.

OUR RESULTS VS THE PAPER



OURS VS PAPER

4 / 4

runs ahead of the paper

- +0.2% to +3.9% higher accuracy
- Lower loss in every run
- Same trend: more classes → lower accuracy

Training curves, 10 categories: validation accuracy levels off near 94% from epoch 4.

Categories	Our accuracy	Our loss	Paper accuracy	Paper loss	Difference
10	95.70%	0.1542	94.60%	0.1815	**+1.10%**
20	93.90%	0.2123	93.70%	0.2235	**+0.20%**
30	92.40%	0.2826	91.10%	0.3521	**+1.30%**
50	92.04%	0.3633	88.18%	0.4284	**+3.86%**

Ours: held-out test set. Paper: validation set (Table I). The paper does not list its categories, so ours differ.

08 · WRAP-UP

TAKEAWAYS

01

Skip connections, H(x) = F(x) + x, beat the vanishing gradient, so ResNet-50 can be 50 layers deep and still learn.

02

Fine-tuned on only 1,000 doodles per class, it reached 95.7% (10 classes) and stayed consistent with more categories: 93.9% (20), 92.4% (30), 92.0% (50).

03

We beat the IEEE ICITCOM 2025 paper in all four runs (by 0.2–3.9%), and saw the same trend: accuracy falls as categories are added.

REFERENCES

[1] K. He, X. Zhang, S. Ren, J. Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016.

[2] B. Alden, N. T. Thung, I. S. Edbert, A. Aulia, “Analyzing Deep Learning in Quick, Draw!: A Case Study on Hand-drawn Sketch Recognition,” IEEE ICITCOM, 2025. doi:10.1109/ICITCOM66635.2025.11265600

[3] Google Creative Lab, “The Quick, Draw! Dataset,” github.com/googlecreativelab/quickdraw-dataset

THANK YOU!